Title: Diving into Windows HTTP: Unveiling Hidden Preauth Vulnerabilities in Windows HTTP Services

Abstract:

The Windows operating system heavily relies on HTTP services. Numerous Windows HTTP services such as IIS, ADFS, ADCS, Hyper-V, Kerberos, UPnP, WSMAN, MSMQ, and RDP are widely deployed and play a crucial role in supporting various core functions within the Windows ecosystem. Although the security of Windows HTTP services is of utmost importance, almost no related security research has been made public in the past. Based on this gap, we decided to dive into the security of Windows HTTP Services and discovered many new things!

After conducting an in-depth analysis of the internal mechanisms of Windows HTTP components, we discovered many novel vulnerability patterns in Windows HTTP services over the past year. These include not only classic memory corruption bugs but also a large number of logical bugs caused by the incorrect usage of Windows HTTP APIs by developers. Our research has identified more than 100 critical pre-auth vulnerabilities in almost all key services, including IIS, ADFS, ADCS, Hyper-V, Kerberos, UPnP, WSMAN, RDP, and MSMQ. These vulnerabilities cover a wide range of issues, including pre-auth remote code execution (RCE), information leakage, and denial-of-service (DoS). Importantly, exploiting these vulnerabilities requires no credentials, no additional configurations, and no user interaction (0-click), which means that any Windows system running them is at risk.

In this presentation, we will discuss the different architectures of Windows HTTP services and share multiple previously undisclosed vulnerability cases and attacks. We will also summarize these new vulnerabilities patterns and provide a comprehensive interpretation of the security threats within the realm of Windows HTTP services.

Agenda

Summary

Logic Flaw Exploration for Pre-auth DoS

1. **CVE-2025-21389 Upnp Device Host Service Unauthenticated Remote Denial of Service Vulnerability - CVSS 7.5**
2. **CVE-2024-38068 Windows Online Certificate Status Protocol (OCSP) Server Unauthenticated Remote Denial of Service Vulnerability - CVSS 7.5**
3. **CVE-2024-43506 BranchCache Unauthenticated Remote Denial of Service Vulnerability - CVSS 7.5**

Parsing and Handling Stage for Pre-auth RCE*

1. **CVE-2024-30080 Windows Message Queueing Unauthenticated Remote Code Execution - CVSS 9.8**
2. **CVE-2025-21309 Windows Remote Desktop Services Unauthenticated Remote Code Execution - CVSS 8.1**

Summary

In this paper, we will showcase the different types of pre-auth RCE and DoS vulnerabilities we have discovered, including their CVE IDs, vulnerability code, and the associated risks.

We list some CVE acknowledgements we found here, Microsoft merged some of them with multiple cases, such as CVE-2024-26197, Microsoft merged 11 different cases in this one CVE, and there are still some cases waiting for patch:

CVE-2024-49106/CVE-2024-49108/CVE-2024-49119/CVE-2024-49120/CVE-2024-49132/CVE-2024-49116/CVE-2024-49128/CVE-2025-21297/CVE-2025-21309/CVE-2025-21278/CVE-2025-21330/CVE-2025-21225/CVE-2024-20655/CVE-2024-20662/CVE-2024-26197/CVE-2024-30080/CVE-2024-30062/CVE-2024-30013/CVE-2024-38031/CVE-2024-38067/CVE-2024-38068/CVE-2024-38230/CVE-2024-43506/CVE-2024-38149/CVE-2024-43512/CVE-2024-43521/CVE-2024-43567/CVE-2024-43575/CVE-2024-49115/CVE-2024-49123/CVE-2024-49129/CVE-2024-49075/CVE-2025-21296/CVE-2025-21389/CVE-2024-43639

Logic Flaw Exploration for Pre-auth DoS

CVE-2025-21389 Upnp Device Host Service Unauthenticated Remote Denial of Service Vulnerability - CVSS 7.5

reference: https://msrc.microsoft.com/update-guide/vulnerability/CVE-2025-21389

The Upnp Device Host service is a typical Windows HTTP service that uses asynchronous processing. This service first calls `HttpReceiveHttpRequest` [1], waiting for the HTTP header request, and uses the `WaitForSingleObjectEx` function to wait for the asynchronous receive signal [2]. This means that when the HTTP request's header section arrives, it triggers the asynchronous receive signal for processing. The `WaitForSingleObjectEx` function returns, and the `GetOverlappedResult` function is used to retrieve the header information.

```
  v9 = (*((__int64 (__fastcall **)(_QWORD, HTTP_REQUEST_ID, _QWORD, struct
_HTTP_REQUEST_V2 *, DWORD, _QWORD, struct _OVERLAPPED *))this // =======> [1]
          + 12))(
           *((_QWORD *)this + 15),
           RequestId,
           0i64,
           v2,
           v4,
           0i64,
           &Overlapped);
    v1 = v9;
    v10 = WPP_GLOBAL_Control;
    if ( WPP_GLOBAL_Control != (CUPnPAutomationProxy *)&WPP_GLOBAL_Control )
    {
      if ( (*((_DWORD *)WPP_GLOBAL_Control + 7) & 0x100) != 0 )
      {
        WPP_SF_D_0(*((_QWORD *)WPP_GLOBAL_Control + 2), 31i64,
&WPP_8928976de11d3d7877ea60ee284ea44a_Traceguids, v9);
        v10 = WPP_GLOBAL_Control;
      }
```

```
      if ( v10 != (CUPnPAutomationProxy *)&WPP_GLOBAL_Control && (*((_DWORD *)v10
+ 7) & 0x100) != 0 )
      {
        WPP_SF_D_0(
          *((_QWORD *)v10 + 2),
          32i64,
          &WPP_8928976de11d3d7877ea60ee284ea44a_Traceguids,
          NumberOfBytesTransferred);
        v10 = WPP_GLOBAL_Control;
      }
    }
    if ( v1 == 997 || !v1 )
    {
      if ( WaitForMultipleObjectsEx(2u, &Handles, 0, 0xFFFFFFFF, 0) != 1 ) //
==================> [2]
      {
        if ( WPP_GLOBAL_Control != (CUPnPAutomationProxy *)&WPP_GLOBAL_Control
          && (*((_DWORD *)WPP_GLOBAL_Control + 7) & 0x100) != 0 )
        {
          WPP_SF_(*((_QWORD *)WPP_GLOBAL_Control + 2), 35i64,
&WPP_8928976de11d3d7877ea60ee284ea44a_Traceguids);
        }
        if ( !CancelIoEx(*((HANDLE *)this + 15), &Overlapped) && GetLastError() ==
1168
          || !WaitForSingleObject(hEvent, 0xFFFFFFFF)
          || WPP_GLOBAL_Control == (CUPnPAutomationProxy *)&WPP_GLOBAL_Control
          || (*((_DWORD *)WPP_GLOBAL_Control + 7) & 0x100) == 0 )
        {
LABEL_84:
          *((_DWORD *)this + 74) = 0;
          if ( v2 )
            free(v2);
          goto LABEL_86;
        }
        v16 = GetLastError();
        if ( v16 > 0 )
          v16 = (unsigned __int16)v16 | 0x80070000;
        v17 = WPP_GLOBAL_Control;
        v18 = 36i64;
        v19 = (unsigned int)v16;
LABEL_83:
        WPP_SF_D_0(*((_QWORD *)v17 + 2), v18,
&WPP_8928976de11d3d7877ea60ee284ea44a_Traceguids, v19);
        goto LABEL_84;
      }
      if ( GetOverlappedResult(*((HANDLE *)this + 15), &Overlapped,
&NumberOfBytesTransferred, 0) ) // ==============> [3]
        v1 = 0;
      else
        v1 = GetLastError();
```

The `HttpReceiveHttpRequest` function is a Windows native API that requires specifying the size of the HTTP header to be received via its parameters(https://learn.microsoft.com/en-us/windows/win32/api/http/nf-http-httpreceivehttprequest). Therefore, if the size of the HTTP request header sent by the client exceeds the specified header size, the function will return `0xEA` [4], which represents `ERROR_MORE_DATA`. When this error is returned, the Upnp Device Host service will retrieve the actual size of the HTTP header [5] and allocate a new buffer to store the header of the required size [6].

```
switch ( v1 )
{
  case 0xEAu: // ===============> [4]
    v4 = NumberOfBytesTransferred; // ==============> [5]
    *((_DWORD *)this + 74) = 0;
    RequestId = v2->RequestId;
    free(v3);
    v13 = (struct _HTTP_REQUEST_V2 *)malloc(v4);
```

During the process from the service receiving the HTTP request and returning `0xEA` to allocating a new buffer for receiving the data, there is a time window. If a malicious attack closes the socket request within this time window, after the service updates the HTTP header buffer, it will call `HttpReceiveHttpRequest` again to receive the full HTTP header. However, due to the malicious attack closing the socket within the time window, the function will return `0x4CD` [6], which represents `ERROR_CONNECTION_INVALID`. In this case, the service does not create a new session to wait for a new HTTP request. Instead, the handler function directly returns [7].

```
case 0x4CDu: // ===============> [6]
          if ( RequestId )
            return; // =============> [7]
```

For this type of HTTP service, the HTTP server creates a corresponding thread when it starts and sets up the handler for processing. However, if the function returns, the service will not proactively create a session. Therefore, when the function returns, it means the service will no longer process HTTP requests, and normal HTTP requests will not be handled. This results in an unauthorized denial-of-service vulnerability.

CVE-2024-43506 BranchCache Unauthenticated Remote Denial of Service Vulnerability - CVSS 7.5

reference: https://msrc.microsoft.com/update-guide/vulnerability/CVE-2024-43506

In the response phase, after the HTTP service has received and parsed the request, it typically responds with a message containing response information, status codes, etc., such as "HTTP 200 OK" or "HTTP 500". Regardless of the type of HTTP service, during the response phase, the underlying Windows Native HTTP API always enters `http.sys` in the kernel for final processing.

During the kernel receive phase, `http.sys` allocates non-paged pool memory for storing HTTP connection-related data by calling `http!UxTlAllocateConnectionForLookaside`. Similarly, after the HTTP response is completed, it calls `http!UxTlFreeConnectionFromLookaside` to free the related objects. The kernel's

disconnect call is triggered by the response phase of the Windows Native HTTP API, with the most typical examples being the `HttpSendHttpResponse` and `HttpCancelHttpRequest` functions.

In the BranchCache service, after data is received, corresponding processing is performed, such as the `GetSegListMsgWorkItem::RunInternal` function [1]. After processing is complete, the `CTnoUpload::SendSegListResponse` function is ultimately called to respond with the SegList content to the client [2]. The function internally calls `HttpSendHttpResponse`, and the client receives the request, while the HTTP server kernel calls disconnect to release memory from the kernel pool.

```
v11 = (struct CritSec **)((char *)this + 392); // =============> [1]
SmartPointer<CTnoDownload>::operator=((char *)this + 392, *Instance);
SmartPointer<CTnoUpload>::Release(v16);
*((_DWORD *)this + 18) = 0;
v6 = v11;
*(_QWORD *)&v10 = CTnoUpload::Complete;
DWORD2(v10) = 0;
v7 = *v11;
v18[0] = 0;
v19 = v7;
v20 = v10;
v21 = 774;
v22 = 1;
InCritSec::InCritSec((InCritSec *)&v11, v7, 1);
CTnoDownloadMgr::RegisterUpload(*v2);
v8 = CTnoDownloadMgr::Authorize(*v2, v15);
CTnoUpload::SetAuthorized(*v6, v8 == 0);
CTnoUpload::Setup(*v6);
CTnoUpload::Start(*v6);
v9 = CTnoUpload::SendSegListResponse(*v6); // ==============> [2]
```

However, when a malicious POST request is sent to BranchCache, after the service successfully receives and parses the malicious POST data, if there is an error in the data, the service will call `ThrowException` for error handling. In the exception handling process, the service does not call `HttpSendHttpResponse` or `HttpCancelHttpRequest` to end the response.

```
   if ( !(unsigned __int8)CTnoDownloadMgr::IsNewInboundRequest(*((_QWORD *)this +
8), &v11) )
  {
    if ( WPP_GLOBAL_Control != (TraceLoggingHProvider)&WPP_GLOBAL_Control
      && (*((_BYTE *)WPP_GLOBAL_Control + 108) & 8) != 0 )
    {
      if ( *((_BYTE *)WPP_GLOBAL_Control + 105) )
        WPP_SF_qqq(
          *((_QWORD *)WPP_GLOBAL_Control + 12),
          675i64,
          &WPP_152a8e42b8b337334125d2feda130716_Traceguids,
          *((_QWORD *)this + 5),
          *v3,
          *((_QWORD *)this + 7));
```

```
    }
    SystemError::ThrowHelper(L"GetSegListMsgWorkItem::RunInternal", -2147024122);
// ================> [3]
```

If the server successfully receives data but does not proactively execute the response operation during subsequent processing, `http.sys` in the kernel will not call the `disconnect` function to release the connection and related structures. As a result, even if the client disconnects from the server, the kernel pool memory will not be released, leading to a memory leak. Since multiple kernel pools are involved in HTTP, when a malicious attacker sends enough requests to the server, it can cause the kernel pool to run out of memory. This will cause the entire system to freeze and become unresponsive. The vulnerability can lead to over 4GB of memory being occupied in the nonpaged pool within a few minutes.

CVE-2024-38068 Windows Online Certificate Status Protocol (OCSP) Server Unauthenticated Remote Denial of Service Vulnerability - CVSS 7.5

reference: https://msrc.microsoft.com/update-guide/vulnerability/CVE-2024-38068

This vulnerability involves the IIS processing mechanism. In essence, IIS is a Windows HTTP service, and it follows and uses the Windows Native HTTP API for HTTP processing. However, it provides a framework that allows other Windows HTTP services or third-party vendors to build their own HTTP servers on top of the IIS framework. When an HTTP service wants to utilize IIS's function calls, it typically uses the `ServerSupportFunction` in `isapi.dll` to execute the corresponding functionality, such as responding to requests (https://learn.microsoft.com/en-us/previous-versions/iis/6.0-sdk/ms524500(v=vs.90)).

IIS creates an object called `ISAPI_CONTEXT` for each HTTP service that uses the HTTP framework to execute server functionality. This object contains a reference count. When an HTTP request for a specific service arrives, IIS increments the reference count. After the processing finishes and the server calls the `ServerSupportFunction` related callback to respond, IIS decreases the reference count. The reference count's upper limit is `0x1366`, meaning that when the request count for a specific HTTP server under IIS reaches `0x1366`, IIS will stop accepting requests for that service. This process is handled in `iiscore.dll`.

In the OCSP service, the client sends domain certificate information via POST data to the OCSP HTTP server. The server first decodes the data [1], then checks if the POST data is valid [2], and finally searches for the domain certificate [3]. When the search fails, the server calls `SendHttpError` to reply with an error message to the client [4].

```
v34 = OcspSvc::OCSPRequestContext::Decode((OcspSvc::OCSPRequestContext *)&v73, *
((_DWORD *)v3 + 366), &v85); // ================> [1]
    v13 = v34;
    if ( v34 < 0 )
    {
      if ( v34 == -2147020560 )
      {
        v87 = L"UnknownSubject";
        CertSrv::CETWEventLog::LogEventStringArrayHResult(
          v36,
          v35,
          v37,
          (const unsigned __int16 *const *)&v87,
```

```
            -2147020560);
          v5 = 6;
        }
        else
        {
          v5 = 1;
        }
        v17 = v13;
        v11 = 135793094;
        goto LABEL_84;
      }
    }
    v38 = OcspSvc::OCSPRequestContext::Validate( // ============================> [2]
            (OcspSvc::OCSPRequestContext *)&v73,
            *((_DWORD *)v3 + 365),
            *((_DWORD *)v3 + 367));
    v13 = v38;
    if ( v38 < 0 )
    {
      v5 = 6;
      v17 = v38;
      v11 = 136579526;
      goto LABEL_84;
    }
    ConditionalVariables = OcspSvc::COcspIsapiExtension::GetConditionalVariables(
                             v39,
                             (struct _EXTENSION_CONTROL_BLOCK *)v8,
                             &v71,
                             &v70);
    v13 = ConditionalVariables;
    if ( ConditionalVariables )
    {
      v17 = ConditionalVariables;
      v11 = 137103814;
      goto LABEL_84;
    }
    Lock = OcspSvc::COCSPRWLock::AcquireReadLock((OcspSvc::COCSPRWLock *)(v3 +
  736));
    v13 = Lock;
    if ( Lock )
    {
      v17 = Lock;
      v11 = 137300422;
      goto LABEL_84;
    }
    v90 = 1;
    v42 = *(_DWORD *)(v77 + 16);
    for ( i = 0; i < v42; ++i )
    {
      if ( v68 )
      {
        OcspSvc::CAHandler::Release(v68);
        v68 = 0i64;
      }
```

```
    v44 = OcspSvc::CAHandlerCollection::Find( // ====================> [3]
            *((PRTL_RESOURCE *)v3 + 178),
            (struct _CRYPTOAPI_BLOB *)(88i64 * i + *(_QWORD *)(v77 + 24) + 24i64),
            (struct _CRYPTOAPI_BLOB *)(88i64 * i + *(_QWORD *)(v77 + 24) + 40i64),
            &v68);
    v13 = v44;
    if ( v44 < 0 )
    {
      v5 = 6;
      v17 = v44;
      v11 = 138676678;
      goto LABEL_84;
    }
  }
LABEL_84:
  goto LABEL_87;
LABEL_87:
  LocalFree(hMem.pbData);
  if ( v68 )
    OcspSvc::CAHandler::Release(v68);
  if ( v6 )
    OcspSvc::COcspResponseCacheEntry::Release(v6);
  if ( v69 )
    OcspSvc::COcspResponseCache::Release(v69);
  if ( v54 )
    OcspSvc::CAHandler::Release(v54);
  if ( v90 )
    RtlReleaseResource((PRTL_RESOURCE)(v3 + 736));
  if ( v72 )
    LocalFree(v72);
  if ( v13 >= 0 )
  {
    v4 = 1;
  }
  else if ( !v5
          || (v62 = OcspSvc::COcspIsapiExtension::SendOCSPStatus(v59, v5), (v63 =
v62) != 0) // ====================> [4]
```

When `SendOCSPStatus` internally calls `ServerSupportFunction`, it first sets the OCSP service's own callback [5]. Then, it calls the function again to respond to the client message [6]. In the `ServerSupportFunction` that responds to the client message, IIS will decrement the reference count of the `ISAPI_CONTEXT` object.

```
if ( (*(unsigned int (__fastcall **)(_QWORD, __int64, void (__stdcall *)(struct
_EXTENSION_CONTROL_BLOCK *, void *, unsigned int, unsigned int), _QWORD, _QWORD
*))(v17 + 184))( // ================> [5]
         *(_QWORD *)(v17 + 8),
         1005i64,
         OcspSvc::COcspIsapiExtension::IOCompletionCallback,
         0i64,
         v14) )
```

```
  {
    if ( (*(unsigned int (__fastcall **)(_QWORD, __int64, char **, _QWORD,
_QWORD))(*((_QWORD *)a1 + 12) + 184i64))( // ===============>[6]
           *(_QWORD *)(*((_QWORD *)a1 + 12) + 8i64),
           1037i64,
           v26,
           0i64,
           0i64) )
    {
      return 0;
    }
```

However, if a malicious client disconnects before the callback is set, the callback setup will fail. If the callback setup fails, the response function will not be called. In other words, the `ISAPI_CONTEXT` reference count will not be decremented. As a result, when the `ISAPI_CONTEXT` reference count reaches 0x1366, IIS will stop accepting HTTP requests, causing all legitimate certificate validation requests to return a "503 Server Unavailable" error. This leads to a permanent denial-of-service attack on the IIS server.

Parsing and Handling Stage for Pre-auth RCE

CVE-2024-30080 Windows Message Queueing Unauthenticated Remote Code Execution - CVSS 9.8

reference: https://msrc.microsoft.com/update-guide/vulnerability/CVE-2024-30080

MSMQ http server service can receive HTTP POST data from any unauthenticated client and then passes it into the `RPCToServer` function. The MSMQ HTTP component acts more like an RPC client; it serializes POST data as parameters of `NdrClientCall3` and sends it to the MSMQ RPC server.

```
CLIENT_CALL_RETURN __fastcall RPCToServer(__int64 a1, __int64 a2, __int64 a3,
__int64 a4)
{
  int v5; // esi
  DWORD LastError; // eax
  int i; // ebx
  __int64 LocalRPCConnection2QM; // rax
  int v12; // [rsp+28h] [rbp-180h]
  int v13; // [rsp+44h] [rbp-164h] BYREF
  __int64 v14; // [rsp+48h] [rbp-160h]
  __int64 v15; // [rsp+50h] [rbp-158h]
  __int64 v16; // [rsp+58h] [rbp-150h]
  CHAR AddressString; // [rsp+60h] [rbp-148h] BYREF
  char v18[271]; // [rsp+61h] [rbp-147h] BYREF

  v5 = a3;
  v14 = a2;
  v15 = a3;
  v16 = a4;
  AddressString = 0;
  memset_0(v18, 0, 0x103ui64);
```

```
  v13 = 260;
  if ( (*(unsigned int (__fastcall **)(_QWORD, const char *, CHAR *, int *))(a1 +
160))(
         *(_QWORD *)(a1 + 8),
         "LOCAL_ADDR",
         &AddressString,
         &v13) )
  {
    if ( WPP_GLOBAL_Control != &WPP_GLOBAL_Control && (*((_BYTE
*)WPP_GLOBAL_Control + 108) & 4) != 0 )
      WPP_SF_s(*((_QWORD *)WPP_GLOBAL_Control + 12));
    for ( i = 0; i <= 1; ++i )
    {
      LocalRPCConnection2QM = GetLocalRPCConnection2QM(&AddressString);
      if ( LocalRPCConnection2QM )
      {
        v12 = v5;
        return NdrClientCall3((MIDL_STUBLESS_PROXY_INFO *)&pProxyInfo, 0, 0i64,
LocalRPCConnection2QM, a2, v12, a4);
      }
      RemoveRPCCacheEntry(&AddressString);
    }
  }
  else if ( WPP_GLOBAL_Control != &WPP_GLOBAL_Control && (*((_BYTE
*)WPP_GLOBAL_Control + 108) & 1) != 0 )
  {
    LastError = GetLastError();
    WPP_SF_D(*((_QWORD *)WPP_GLOBAL_Control + 12), 32i64,
&WPP_bc048c6c4e693d2cb11110aaa1848a3f_Traceguids, LastError);
  }
  return 0i64;
}
```

In the `GetLocalRPCConnection2QM` function, the service retrieves the RPC binding handle from a global variable. If the global variable is empty, it first binds the handle to the RPC server and then returns to the outer function.

In the `RemoveRPCCacheEntry` function, it removes the RPC binding handle from the global variable and then invokes `RpcBindingFree` to release the RPC binding handle.

A typical RPC client defines an IDL file to specify the type of parameter for the RPC interface. When invoking `NdrClientCall3`, the parameters are marshalled according to the IDL. If the parameter is invalid, it will crash the RPC client when it is serialized in rpcrt4.dll. This is why we sometimes encounter client crashes when hunting bugs in the RPC server.

To prevent client crashes, RPC server usually add RPC exceptions in the code as follows:

```
    RpcTryExcept
    {
        [...]
    }
```

```
    RpcExcept(1)
    {
        ULONG ulCode = RpcExceptionCode();
        printf("Run time reported exception 0x%lx = %ld\n",
            ulCode, ulCode);
        return false;
    }
    RpcEndExcept
        return true;
```

It's clear now that the overlooked pattern is that the `NdrClientCall3` function is within an RPC exception. This means if an unauthenticated user passes an invalid parameter into `NdrClientCall3`, it triggers a crash during marshalling in rpcrt4.dll, which then invokes the `RemoveRPCCacheEntry` function to release the RPC binding handle as it will be invoked in `RpcExcept`.

There is a time window where if one thread passes an invalid parameter and then releases the RPC binding handle, while another thread retrieves the RPC binding handle from the global variable and passes it into `NdrClientCall3`, it will use the freed RPC handle inside rpcrt4.dll.

This causes an use after free situation leading to unauthenticated remote code execution.

CVE-2025-21309 Windows Remote Desktop Services Unauthenticated Remote Code Execution - CVSS 8.1

reference: https://msrc.microsoft.com/update-guide/vulnerability/CVE-2025-21309

Remote Desktop Gateway is a gateway for RDP service. When a new connection established while another thread is processing disconnection for another connection, the receive callback will compete with disconnect operation and access `CAAHttpServerConnection` structure which has been freed, this will cause an UAF problem which may cause RCE.

When a request comes from https, it will call `CAAHttpServerTransport::ProcessOutChannelOrWebSocketRequest` to create a **connection_info** structure.

```
CAAHttpServerTransport::ProcessOutChannelOrWebSocketRequest()
{
  ......
  AcquireSRWLockShared(this->lock_70h);
  v152 = &this->field_78;// ConId provided by Client requests.
  v18 = std::wstring::wstring(&v167, a2.ConId);
  _hash_get(&this->field_78, &v145, v18);
  std::wstring::_Tidy(&v167, 1, 0i64);
  ReleaseSRWLockShared(this->lock_70h);
  if ( v145 != this->field_78.qword0 )
  {
      ......
  }
  .......
  var = operator new(0x958ui64);
```

```
  connection_info =
CAAHttpServerTransport::HTTP_TRANSPORT_CONNECTION_INFO::HTTP_TRANSPORT_CONNECTION_
INFO(var);
  ......
  v54 = LocalAlloc(0x40u, 0xC0ui64);
  connection_info->heap1_C0h_6C0h = v54;
  ......
  AcquireSRWLockExclusive(SRWLock);
  *hash_add(v152, v99) = connection_info;
  ......
  ReleaseSRWLockExclusive(v100->lock_70h);
  ......
}
```

In this function, it firstly tries to get the **connection_info** structure from the hash map by `ConId` (comes from client) from line 4 to 9.

If it doesn't exist, it will allocate a new **connection_info** structure at line 15,16. Then, it allocates a heap(mark as **heap1**) for http receive callback at line 18,19. Later, it adds the new connection_info into the hash map at line 22.

If connection is disconnected, it will remove the `connection_info` from the hash table and then call function `CAAHttpServerTransport::CleanupConnInfo`:

```
void __fastcall CAAHttpServerTransport::CleanupConnInfo(
        CAAHttpServerTransport *this,
        struct HTTP_TRANSPORT_CONNECTION_INFO *a2)
{
    ......

  v10 = *(_QWORD *)(a2 + 0x688);                //  point to
CAAHttpServerConnection+10h
  if ( v10 )
  {
    v11 = v10 + 8 + *(int *)(*(_QWORD *)(v10 + 8) + 4i64);
    (*(void (__fastcall **)(__int64))(*(_QWORD *)v11 + 8i64))(v11);// release the
reference of CAAHttpServerConnection
    *(_QWORD *)(a2 + 1672) = 0i64;
  }
  v12 = *(_QWORD *)(a2 + 0x690);                //  point to
CAAHttpServerConnection+20h
  if ( v12 )
  {
    v13 = v12 + 8 + *(int *)(*(_QWORD *)(v12 + 8) + 4i64);
    (*(void (__fastcall **)(__int64))(*(_QWORD *)v13 + 8i64))(v13);// release the
reference of CAAHttpServerConnection
    *(_QWORD *)(a2 + 0x690) = 0i64;
  }
    ......
}
```

As you can see, it will release the references of `CAAHttpServerConnection`. And `CAAHttpServerConnection` will be freed after it.

In `CAAHttpServerConnection::OnConnected`, it will call `CAAHttpServerTransport::ReceiveData`.

```
  CAAHttpServerTransport_13E0h = this->f_10h.f_30h.CAAHttpServerTransport_13E0h;
  this->f_10h.f_30h.conn_state_1410h = 2;
  this->f_10h.f_30h.count_size_7430h = 0;
  ret = CAAHttpServerTransport_13E0h->qword0-
>func_ReceiveData_CAAHttpServerTransport_1800814e0_30h(//
                                                // 1800814e0
          CAAHttpServerTransport_13E0h,
          (unsigned __int16 *)&this->f_10h.f_30h.conId_5Ch,
          this->f_10h.f_30h.websocekt_recv_buf_1430h,
          0x6000u,
          14u,
          0i64);
```

As you can see in the upon code, it calls `CAAHttpServerTransport::ReceiveData` by a function pointer, and use `CAAHttpServerConnection->f_10h.f_30h.websocekt_recv_buf_1430h` as argument 3.

In `CAAHttpServerTransport::ReceiveData`:

```
  qword30->heap1_C0h_6C0h->de0_88h = a6;
  qword30->heap1_C0h_6C0h->buff_90h = a3;
  qword30->heap1_C0h_6C0h->bufsize_6000_98h = buf_size;
  qword30->heap1_C0h_6C0h->min_size_B8h = a5.request_size;
  qword30->heap1_C0h_6C0h->get_size_BCh = 0;
  heap1_C0h_6C0h = qword30->heap1_C0h_6C0h;
```

it will assign argument3 `a3` to field `90h` of **heap1**.

In `CAAHttpServerTransport::WebSocketReceiveLoop`:

```
__int64 __fastcall CAAHttpServerTransport::WebSocketReceiveLoop(
        CAAHttpServerTransport *this,
        websocket_server *a2,
        unsigned __int64 a3.reqId,
        int a4)
{
  ......
  while ( 1 )
  {
    v10 = WebSocketGetAction(
        a2->web_socket_handle__10h,
        2i64,
        &Source,
        &web_buf_num,
```

```
        &action,
        &type,
        &AppCtx,// should be heap1 pointer
        &actionctx);
    if (type != 4){
        ......
    }
    ......
    AcquireSRWLockShared((PSRWLOCK)v7->lock_70h);
    v25 = std::wstring::wstring((wstring *)&ActivityId, (char *)&heap1->field_38);
    _hash_get(&v7->field_78, &a2a, v25);
    std::wstring::_Tidy((wstring *)&ActivityId, 1, 0i64);
    if ( a2a == v7->field_78.qword0 ){
        ......
        LocalFree(AppCtx);
        ......
    }
    connection_info = a2a->qword30;
    ......
    if ( type != 0x80000002 )
    {
      if ( type != 0x80000003 )
      {
        connection_info->flag_inuse_receive_6BCh = 0;
        ReleaseSRWLockShared(this->lock_70h);
        ......
    }
    ......
    if ( memcpy_s((heap1->buff_90h + heap1->get_size_BCh), v38,
websocket_buffers.pbBuffer, recv_size) )
    .....
}
```

In this function, it calls `WebSocketGetAction` to get websocket action and AppCtx at line 10. And AppCtx should be **heap1**.

Then, it tries to get **connection_info** structure from the hash table from line 23 to 27. If it exists, it will assign the structure pointer to variable `connection_info` at line 32. if `type` is 0x80000002, it will do a `memcpy` at line 43 and copy data to **heap1->buff_90h**.

Now, think about this situation:

1. **socket1** sends a request to server, the server registers a disconnect callback and a receive callback for socket1, and creates `connection_info1` and add it into hash map by `ConId1`.
2. **socket1** is closed. the server calls disconnect callback to remove `connection_info1` from the hash map by `ConId1`, and call `CAAHttpServerTransport::CleanupConnInfo` to free `CAAHttpServerConnection`.
3. Meanwhile, **socket2** sends a request to server with same connection id:`ConId1`, in the thread1, the server creates `connection_info2` and add it into hash map by `ConId1`.

4. In the thread2, **the receive callback for socket1 is running**. it gets `connection_info2` from hash map by `ConID1` in the function `CAAHttpServerTransport::WebSocketReceiveLoop`. Then it runs to `memcpy` and try to copy data into **heap1->buff_90h**. However, the `buff_90h` comes from structure `CAAHttpServerConnection`, and it has been freed in step 2.

Thus, it will write data into a freed buffer.

Thus, this could cause unauthenticated RCE issue.

Conclusion

In the draft, we briefly outline the types of vulnerabilities, vulnerability codes, and causes related to Windows HTTP services. These vulnerabilities require no additional configuration or authentication; they can be triggered and exploited simply by crafting a carefully designed HTTP request. In the presentation, we will delve deeper into the operating mechanisms of Windows HTTP services, as well as provide more details on undisclosed unauthorized remote vulnerabilities.